



AHB-Lite Master Project

Professional Hardware Design Document

Project Name: AHB-Lite Master IP

Author: Bassel Mahmoud

Date: July 21, 2026

Contents

1	Introduction	1
2	AHB-Lite Overview	1
2.1	Address Phase	1
2.2	Data Phase	1
2.3	Wait States	1
2.4	Burst Transfers	1
2.5	HREADY	1
2.6	HRESP	2
2.7	HTRANS	2
3	Project Assumptions	3
4	Top Level Architecture	4
4.1	Top Block Diagram	4
5	Previous Block Interface	5
5.1	Block Diagram	5
5.2	Signal Table	5
5.3	Timing	6
6	Slave Interface	7
6.1	Block Diagram	7
7	Master Interface	8
7.1	Block Diagram	8
8	Internal Architecture	9
8.1	Pipeline Registers	9
8.2	State Register	9
8.3	Address Generation	9
8.4	Burst Counter	9
8.5	Control Register Updates	9
8.6	Registered Outputs	9
9	Burst Counter	10

9.1	Fixed Bursts	10
9.2	INCR Bursts	10
9.3	Termination	10
10	Address Generation	10
11	FSM	11
11.1	FSM Chart	11
11.2	IDLE	11
11.3	NONSEQ	11
11.4	SEQ	11
11.5	BUSY	12
11.6	Moore Outputs	12
12	Wait State Handling	12
12.1	Frozen Signals	12
12.2	Exceptions	12
12.3	Behavioral Note	13
13	Output Logic	13
13.1	ctrl_update_en	13
14	Data Pipeline	14
14.1	Address Phase	14
14.2	Data Phase	14
14.3	Why data_reg Exists	14
15	Address Decoder	15
15.1	Selection Rule	15
15.2	Formula	15
16	Multiplexer	17
16.1	Routing	17
17	Limitations	17
18	Future Improvements	17
19	Conclusion	18

A Enumerations	18
A.1 HTRANS	18
A.2 HSIZE	18
A.3 HBURST	19
A.4 State Encoding	19
B Important Formulas	19
B.1 Address Increment	19
B.2 Remaining Beats	19
B.3 Transfer Acceptance	19
B.4 Burst Completion	19

1 Introduction

Advanced High-performance Bus Lite (AHB-Lite) is the single-master subset of the AMBA AHB family. It is designed for synthesizable on-chip interconnects that need a fixed pipeline between address and data phases, predictable timing, and high throughput without the complexity of full AHB arbitration.

This project implements an AHB-Lite master that accepts transaction requests from an upstream request generator, drives the AHB address and control channels, and returns read data and bus-ready feedback to the requester. The design is optimized for a single-master system with a simple interconnect and a set of memory-mapped slaves.

2 AHB-Lite Overview

AHB-Lite uses a pipelined transfer model with separate address and data phases. The master places a transfer address and control information on the bus during one cycle, while the data phase of the previous transfer completes in the next cycle. This overlap increases throughput compared with non-pipelined buses.

2.1 Address Phase

The address phase carries HADDR, HTRANS, HSIZE, HBURST, HWRITE, HPROT, and HMASTLOCK. "Note HMASTLOCK and HPROT isn't supported as functional Signals in this design". The master presents a new address phase when the bus is ready to accept updated control.

2.2 Data Phase

The data phase carries write data on HWDATA and read data on HRDATA. For reads, the addressed slave drives HRDATA. For writes, the master drives HWDATA.

2.3 Wait States

When a slave deasserts HREADY, the current transfer is extended. The master must hold the address and control outputs stable until the slave completes the transfer. The implemented master also exposes a **ready** output to the upstream block so that Transaction generation can be stalled "No extra request till the master is ready to proceed with next Transaction" .

2.4 Burst Transfers

The design supports single transfers, incrementing bursts, and fixed-length bursts **INCRx** . Incrementing bursts are terminated using the **end_burst** input from the previous block, which tells the master when the upstream transaction stream has ended.

2.5 HREADY

HREADY indicates whether the slave has completed the transfer. When it is low, the bus remains in the current transfer phase. The master uses this to freeze the control pipeline and the burst counter.

2.6 HRESP

HRESP communicates the slave response. This implementation treats the response as either OKAY or ERROR. Error handling follows the two-cycle AHB-Lite convention.

2.7 HTRANS

HTRANS encodes the current transfer type. The master drives IDLE, NONSEQ, SEQ, and BUSY as needed by the request stream and burst state.

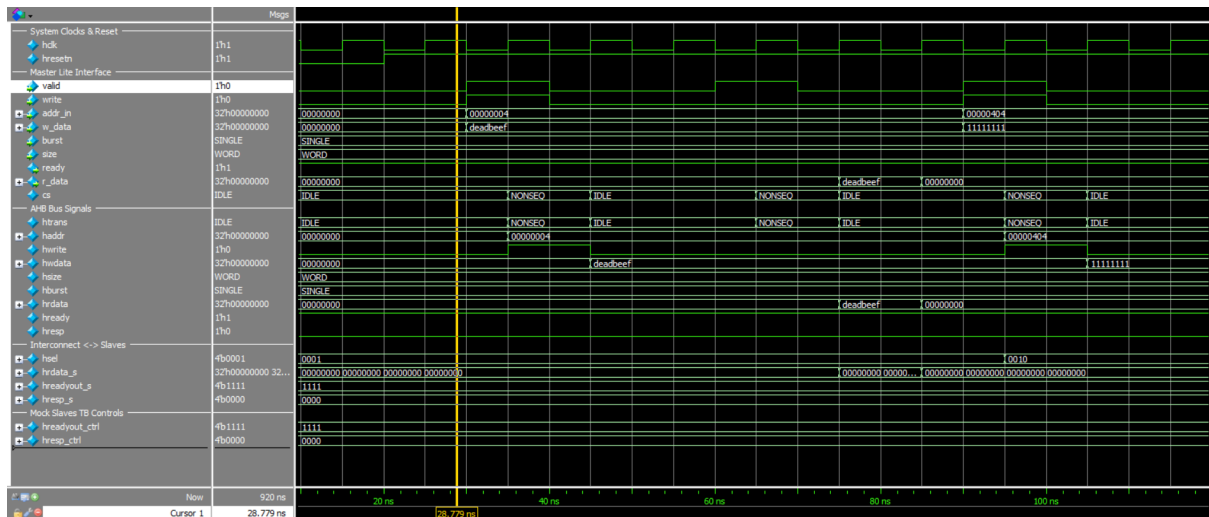


Figure 1: AHB-Lite Pipeline

3 Project Assumptions

The design intentionally constrains the supported feature set to match the implemented RTL.

Assumptions and Rationale

Assumption	Reason
Single master system (Lite-edition)	The RTL exposes one master interface and does not include arbitration logic.
Multiple slaves behind a decoder	The interconnect routes the master address to one selected slave at a time.
Reset is synchronous deasserted and asynchronous asserted	The master follows the AHB-Lite reset convention and drives legal idle control values during reset.
No WRAP burst support	WRAP bursts are forced to IDLE in the master output logic.
No locked transfers	HMASTLOCK is hardwired low.
Fixed HPROT	HPROT is hardwired to 4'b0011 (default value).
Undefined bursts use end_burst	The master needs an external end-of-stream indicator for INCR bursts.
Address increments by transfer size	The design updates burst addresses using the encoded transfer width.
Only OKAY and ERROR responses are used	The implemented response path does not include alternative termination codes.

Design Assumption

If a feature is listed as unsupported, it is not hidden elsewhere in the RTL; it is intentionally absent from the implementation.

4 Top Level Architecture

The top level system contains the previous block, the AHB-Lite master, an address decoder, a multiplexer, and one or more slaves.

The previous block provides transaction intent. The master converts that intent into bus-level control signals. The decoder selects the target slave based on the current address. The multiplexer returns the selected slave's HRDATA, HREADY, and HRESP to the master.

4.1 Top Block Diagram

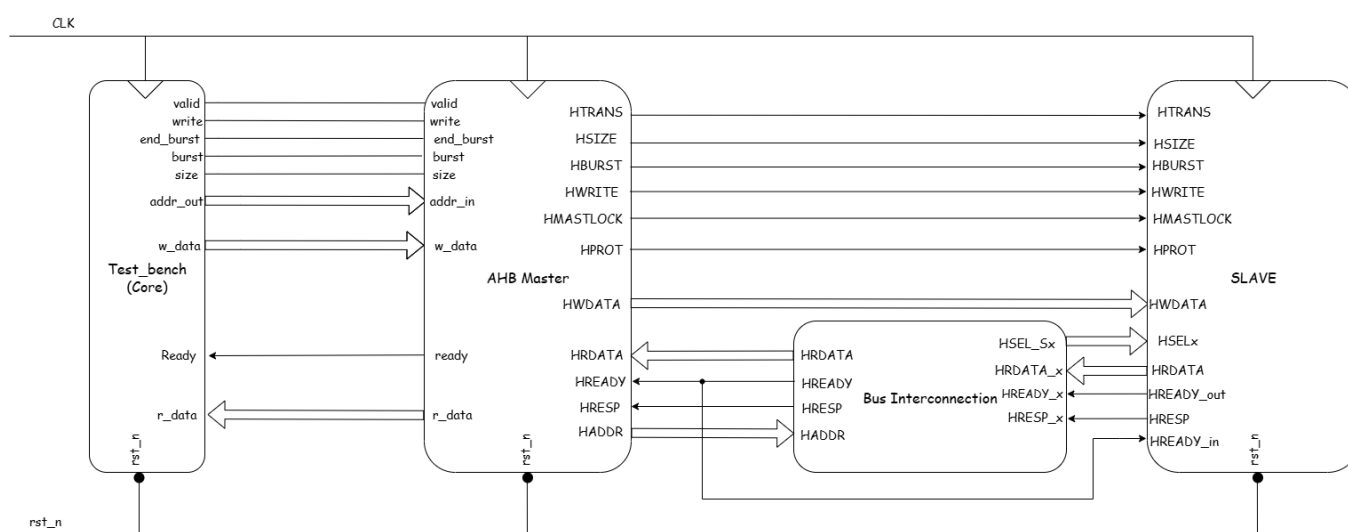


Figure 2: Top Block Diagram

Design Assumption

The interconnect shown here reflects the implemented single-address-decoder, single-response-multiplexer organization. A more complex matrix or multi-master fabric is outside the scope of the current RTL.

5 Previous Block Interface

The previous block is the request source for the master. It asserts `valid` when a transfer request is available and provides the associated metadata and data payload.

5.1 Block Diagram

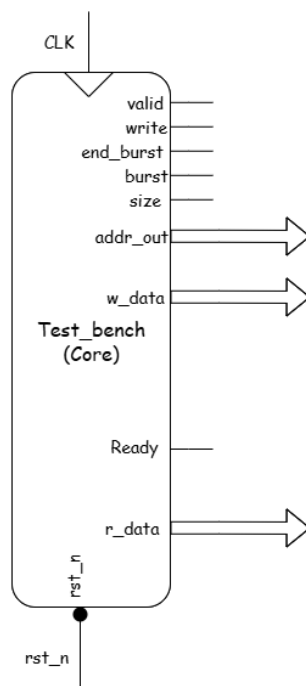


Figure 3: Tb Block Diagram

5.2 Signal Table

Table 2: Previous Block Interface

Signal	Direction	Width	Description
valid	In	1	Indicates that a transaction is available and should be accepted when the bus allows it.
write	In	1	Selects write transfers when high and read transfers when low.
size	In	enum	Encodes the transfer size requested by the upstream block.
burst	In	enum	Encodes the requested burst type.
addr_in	In	ADDR_WIDTH	Start address for the current transfer or burst.
w_data	In	DATA_WIDTH	Write data for write transfers.
end_burst	In	1	Terminates an undefined-length INCR burst.

5.3 Timing

The master samples request inputs when `ctrl_update_en` is asserted. In the normal case this occurs when `HREADY` is high. Two special cases allow control to advance even when the slave is making Master wait: an IDLE-to-NONSEQ request and a BUSY-to-SEQ continuation. The design also advances control on the first cycle of an ERROR response, matching the two-cycle error sequence behavior.

Design Assumption

The upstream block must hold request fields stable while the master isn't ready to accept new transaction.

6 Slave Interface

The slave side provides the completion status and read data for the currently selected slave.

6.1 Block Diagram

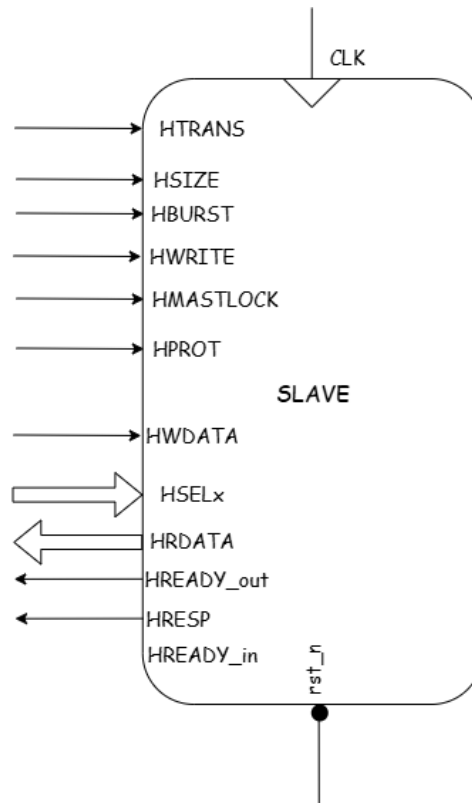


Figure 4: Slave Block Diagram

Table 3: Slave Interface Signals

Signal	Direction	Width	Description
HREADY	In	1	Indicates that the selected slave has completed the transfer.
HRESP	In	1	Indicates OKAY when low or ERROR when high.
HRDATA	In	DATA_WIDTH	Read data returned by the selected slave.

Slave wait states are modeled by deasserting HREADY. The master freezes its control outputs in response. Slave error behavior follows the ARM two-cycle convention: the first cycle of ERROR is issued with HREADY=0, then the second cycle completes with HREADY=1 and HRESP=1.

7 Master Interface

The master exports the bus signals to the interconnect and returns the read data and ready indication to the request source.

7.1 Block Diagram

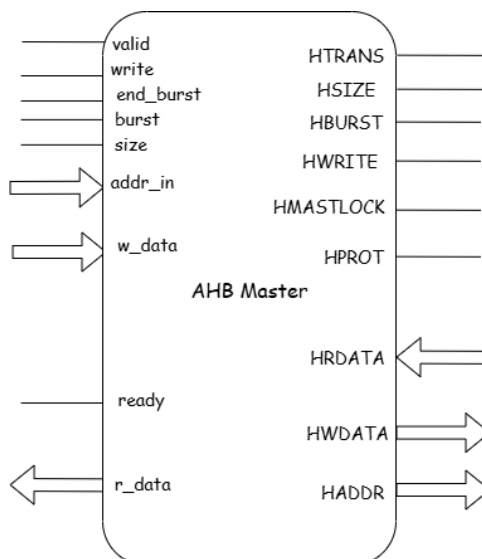


Figure 5: AHB MASTER Block Diagram

Table 4: Master Outputs

Signal	Direction	Width	Description
HADDR	Out	ADDR_WIDTH	Address bus driven by the master.
HTRANS	Out	2	Transfer type for the current address phase.
HWDATA	Out	DATA_WIDTH	Write data for the data phase.
HSIZE	Out	3	Encoded transfer size.
HBURST	Out	3	Encoded burst type.
HWRITE	Out	1	Transfer direction, 1 for write and 0 for read.
HMASTLOCK	Out	1	Hardwired low in this implementation.
HPROT	Out	4	Hardwired protection code 4'b0011.
ready	Out	1	Back-pressure or acceptance indication to the previous block.
r_data	Out	DATA_WIDTH	Read data returned from the selected slave.

Design Assumption

The master interface is intentionally minimal. Signals such as **HBUSREQ**, **HGRANT**, or multi-master arbitration inputs are not implemented because the design targets AHB-Lite rather than full AHB.

8 Internal Architecture

The implementation is organized around a small set of pipeline registers and control registers.

8.1 Pipeline Registers

The `data_reg` register captures write transactions so that the write data can be aligned with the data phase. The `hwdata` register then drives the bus when the write data phase is active.

8.2 State Register

The `cs` register stores the current FSM state and is updated on every rising edge of `HCLK`. The next state logic is purely combinational.

8.3 Address Generation

The master computes a byte increment based on the registered `HSIZE`. The address advances only in the `SEQ` state. In `IDLE` or `BUSY` the address is held constant.

8.4 Burst Counter

The `remaining_beats` counter tracks the number of transfers left in fixed-length bursts. It is loaded on the first `NONSEQ` beat and decremented on each accepted `SEQ` beat.

8.5 Control Register Updates

The control outputs update only when `ctrl_update_en` is asserted. This prevents the bus from changing mid-transfer during wait states, while still allowing protocol-required exception paths.

8.6 Registered Outputs

The master registers `HADDR`, `HTRANS`, `HWRITE`, `HSIZE`, `HBURST`, `HPROT`, and `HMASTLOCK`. This keeps timing predictable and ensures that the bus control phase is synchronous.

9 Burst Counter

The burst counter is used for bursts with a known length. The RTL maps burst encodings to lengths as follows: SINGLE=1, INCR=0, WRAP4/INCR4=4, WRAP8/INCR8=8, and WRAP16/INCR16=16.

9.1 Fixed Bursts

For fixed bursts, `remaining_beats` is loaded with `burst_len - 1` after the first beat is accepted. The counter decrements on each subsequent accepted SEQ beat.

9.2 INCR Bursts

For INCR bursts, the length is undefined, so the burst counter is not used for termination. Instead, the upstream block asserts `end_burst` when the sequence should end.

9.3 Termination

Termination occurs when the last fixed burst beat is accepted or when `end_burst` is asserted for an undefined burst. This allows the master to continue incrementing until the upstream transaction source decides to stop.

10 Address Generation

The address generator increments the current address by the number of bytes implied by `HSIZE`. The implemented mapping is:

$$\text{next_address} = \text{current_address} + \text{transfer_size}$$

where `transfer_size` is decoded from `HSIZE` as 1, 2, 4, 8, 16, 32, 64, or 128 bytes.

11 FSM

The finite state machine controls the transfer type and burst progression. The implemented states are IDLE, NONSEQ, SEQ, and BUSY .

11.1 FSM Chart

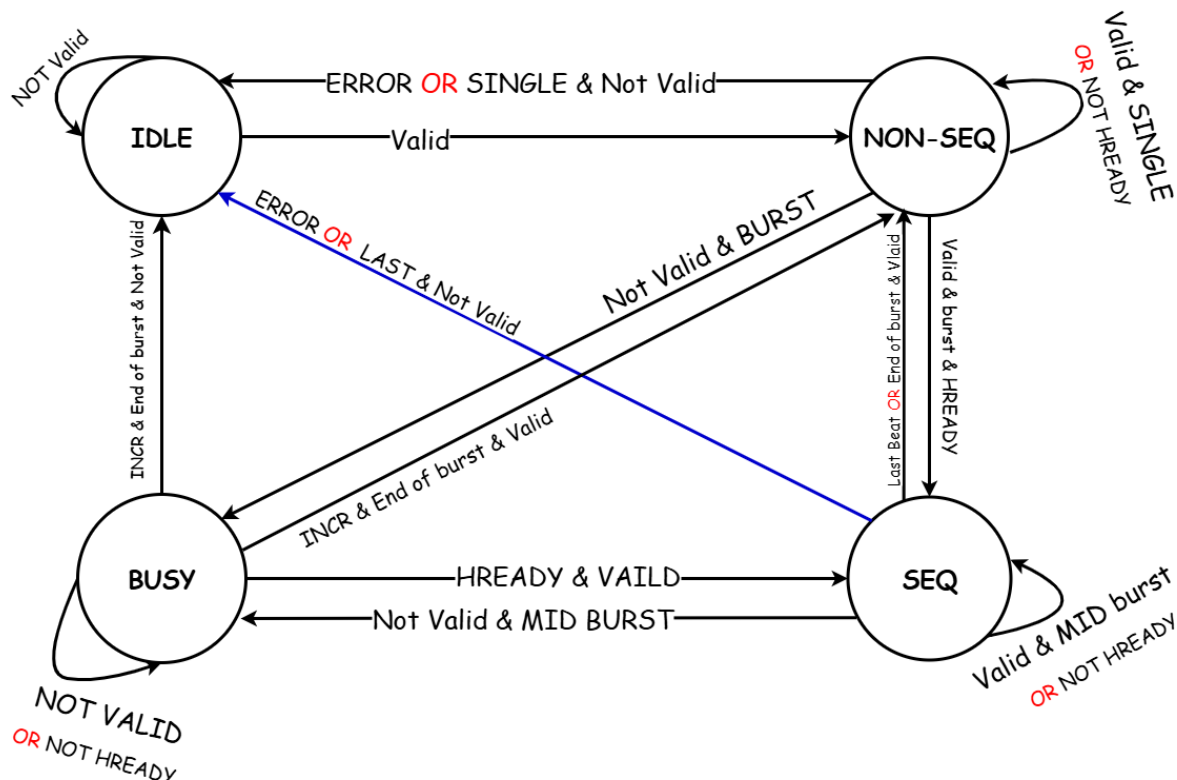


Figure 6: Master FSM

11.2 IDLE

IDLE is the Reset state. When `valid` is asserted, the FSM moves to NONSEQ and starts a new transfer.

11.3 NONSEQ

NONSEQ marks the first transfer of a transaction or burst. If the request is accepted and more beats remain, the FSM transitions to SEQ or BUSY depending on whether the upstream block is already presenting the next beat.

11.4 SEQ

SEQ represents a continuing burst transfer. The FSM remains in SEQ while more beats are available and the slave is ready. If the burst ends, it transitions back to NONSEQ for the next transaction or to IDLE if no more work is pending.

11.5 BUSY

BUSY exists to represent a stall in the request stream during a burst while the bus remains occupied. It is especially useful when the bus is ready but the upstream block has not yet provided the next beat of a fixed or incrementing burst.

11.6 Moore Outputs

The state machine is Moore-like in the sense that HTRANS is generated from the next state calculation rather than from the current combinational request alone.

Design Assumption

The BUSY state is retained because it preserves burst context without forcing the upstream block to present the next beat immediately.

12 Wait State Handling

When HREADY=0, the current transfer is extended. The master therefore freezes all bus control outputs that define the transfer context.

12.1 Frozen Signals

The following signals remain stable during a wait state:

- HADDR
- HTRANS
- HSIZE
- HBURST
- HWRITE
- HPROT
- HMASTLOCK
- HWDATA for write transfers

12.2 Exceptions

The RTL explicitly allows control advancement during:

- IDLE to NONSEQ while HREADY=0
- BUSY to SEQ while HREADY=0
- The first cycle of an ERROR response

12.3 Behavioral Note

These exceptions align the request pipeline with the bus protocol and ensure that legal address/control changes can still occur at the right boundary when a stalled transfer completes or when the first cycle of an error response is accepted.

Design Assumption

The upstream request source is expected to understand that a low `ready` value means the current request has not yet advanced and it shouldn't pass the new transaction since it won't be processed.

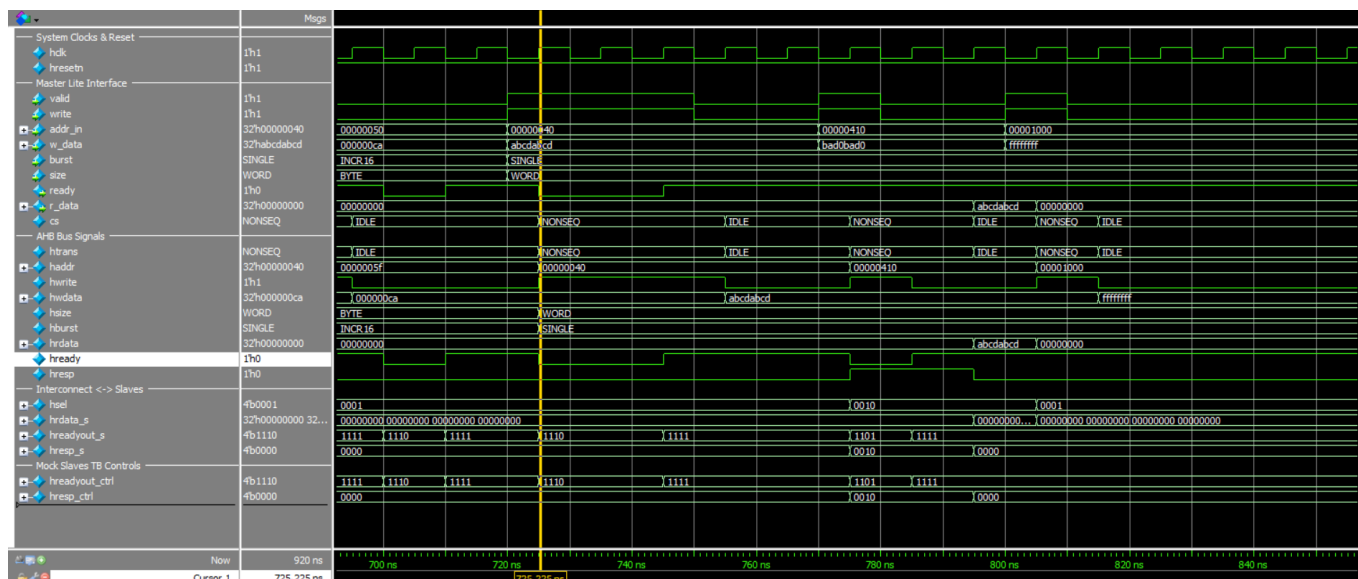


Figure 7: Wait State Timing

13 Output Logic

The output logic is registered. This reduces combinational depth, improves timing closure, and ensures that address/control outputs only change when the design has determined that a legal update is allowed.

13.1 ctrl_update_en

The enable is asserted when:

- HREADY is high, or
- the design is transitioning from IDLE to NONSEQ while wait state, or
- the design is transitioning from BUSY to SEQ while wait state, or
- the design is handling the first cycle of ERROR.

14 Data Pipeline

The data path separates request data capture from the actual bus drive.

14.1 Address Phase

The address phase is driven by the currently active control registers. For write transactions, the associated payload must already be available or latched internally.

14.2 Data Phase

`data_reg` captures the write payload when `HREADY` and `valid` and `write` are all high. The bus-facing `HWDATA` register then sources the actual write data when `hwrite` is asserted.

14.3 Why data_reg Exists

This register decouples upstream request timing from the AHB data phase. It ensures that the write payload is preserved even if the slave inserts wait states.

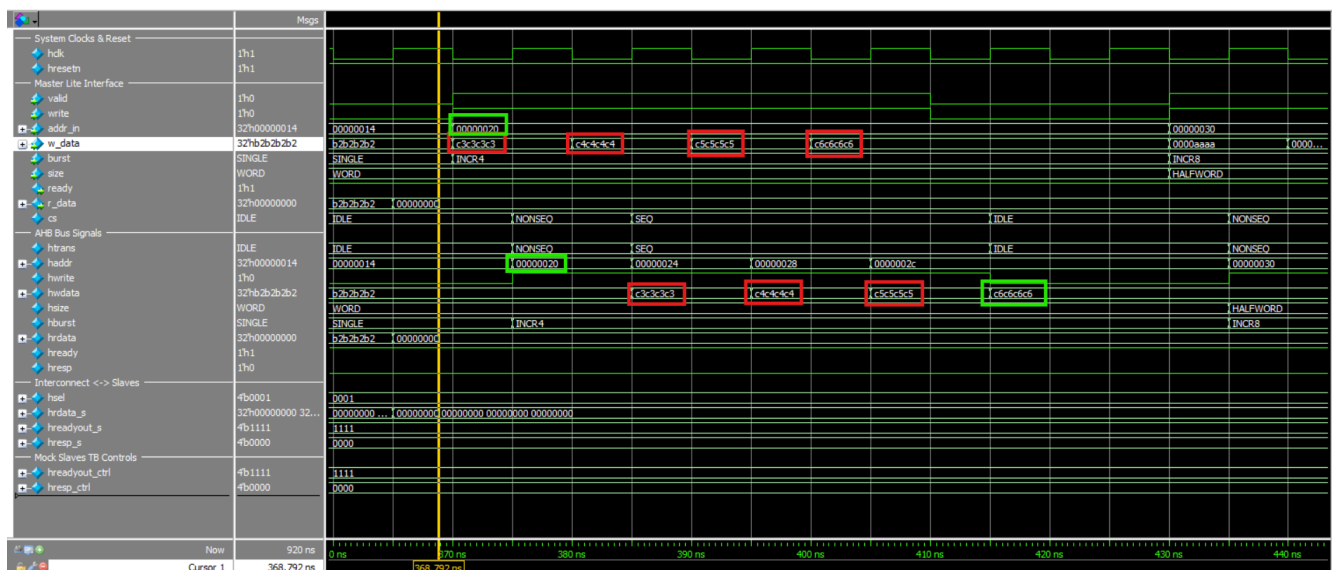


Figure 8: Data Pipeline

15 Address Decoder

The interconnect decoder derives the slave select signal from the upper address bits. The address space is divided into equal-sized regions based on `SLAVE_SIZE_KB`. The selected slave index is extracted from the address slice beginning at the byte offset determined by the slave region size.

15.1 Selection Rule

If the decoded index is within range, exactly one `HSEL` bit is asserted. If the address is outside the supported slave range, **Default slave** "First Slave" is selected and the interconnect returns an error response.

15.2 Formula

Let

$$\text{offset} = \log_2(\text{SLAVE_SIZE_KB} \times 1024)$$

and let `slave_index` be the address bits starting at `offset`. Then:

$$\text{HSEL}[\text{slave_index}] = 1$$

provided that `slave_index < SLAVE_NUM`.

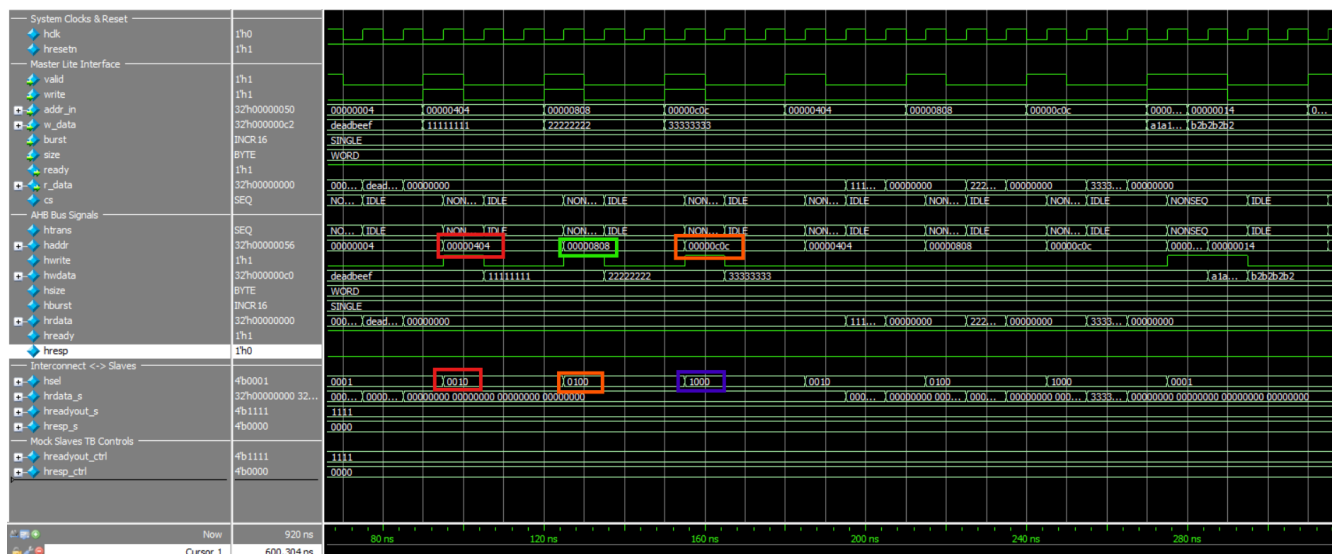


Figure 9: Address Decoder

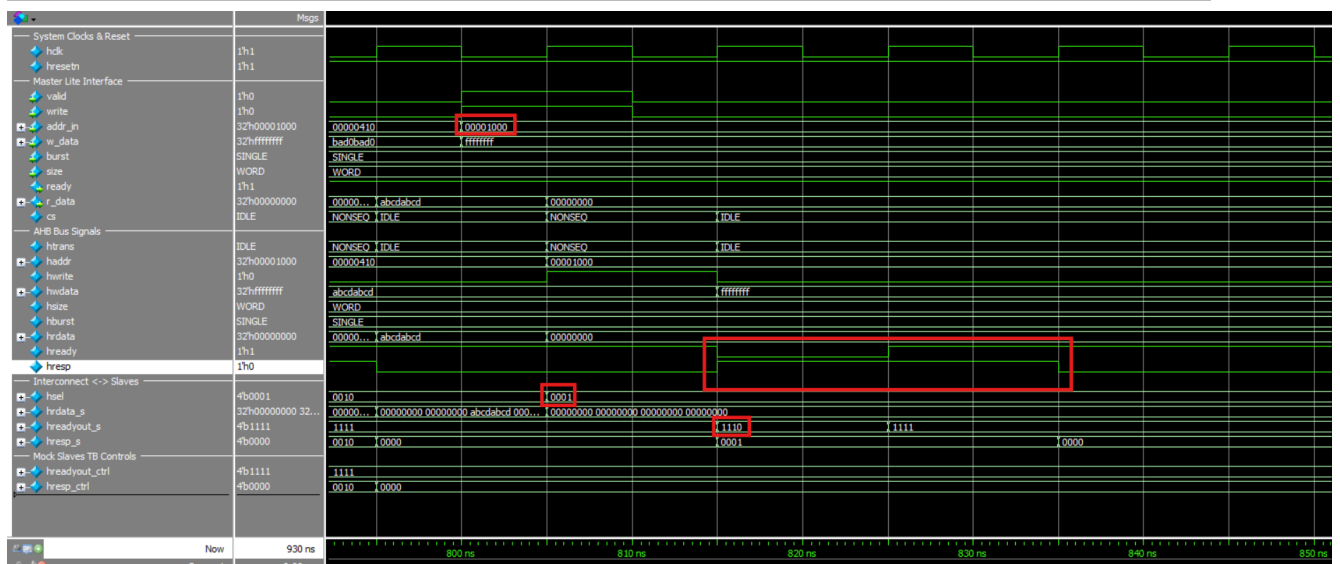


Figure 10: Wrong Address Handling

16 Multiplexer

The multiplexer returns the selected slave's response signals to the master. The choice of HRDATA, HREADY, and HRESP is based on the decoded slave index.

16.1 Routing

When the address decodes to a valid slave, the interconnect routes:

- HRDATA from the selected slave's data bus
- HREADY from the selected slave's ready output
- HRESP from the selected slave's response output

Design Assumption

The interconnect is purely combinational and assumes the selected slave's outputs are already valid for the current address phase.

17 Limitations

The current implementation does not support:

- WRAP bursts
- Locked transfers
- Exclusive transfers
- Multi-master arbitration
- Bus matrix interconnect behavior

18 Future Improvements

Natural follow-on work includes:

- Add WRAP burst boundary logic.
- Extend the design to multiple masters with arbitration.
- Replace the simple interconnect with a bus matrix.
- Add timeout detection for hung slaves.
- Strengthen verification with assertions and formal properties.
- Improve decoder robustness for system-level integration.

19 Conclusion

This project implements a compact AHB-Lite master with a simple, practical feature set. It provides registered control outputs, burst-aware address generation, wait-state handling, fixed-response routing through a combinational interconnect, and explicit integration assumptions for undefined bursts and unsupported protocol features. The result is a maintainable master IP block that is suitable for a small single-master AHB-Lite subsystem.

A Enumerations

A.1 HTRANS

Table 5: HTRANS Encoding

Value	Meaning
2'b00	IDLE
2'b01	BUSY
2'b10	NONSEQ
2'b11	SEQ

A.2 HSIZE

Table 6: HSIZE Encoding

Value	Meaning
3'b000	BYTE
3'b001	HALFWORD
3'b010	WORD
3'b011	DOUBLEWORD
3'b100	WORD_4
3'b101	WORD_8
3'b110	BITS_512
3'b111	BITS_1024

A.3 HBURST

Table 7: HBURST Encoding

Value	Meaning
3'b000	SINGLE
3'b001	INCR
3'b010	WRAP4
3'b011	INCR4
3'b100	WRAP8
3'b101	INCR8
3'b110	WRAP16
3'b111	INCR16

A.4 State Encoding

Table 8: FSM State Encoding

Value	Meaning
2'b00	IDLE
2'b01	BUSY
2'b10	NONSEQ
2'b11	SEQ

B Important Formulas

B.1 Address Increment

$$\text{next_address} = \text{current_address} + \text{transfer_size}$$

B.2 Remaining Beats

For fixed bursts:

$$\text{remaining_beats}_{n+1} = \text{remaining_beats}_n - 1$$

B.3 Transfer Acceptance

A transfer is accepted when the bus is ready and control is allowed to update:

$$\text{accepted} = \text{ctrl_update_en}$$

B.4 Burst Completion

An undefined burst completes when:

$$\text{burst} = \text{INCR} \wedge \text{end_burst} = 1$$